A D PATEL INSTITUTE OF TECHNOLOGY
(The Charutar Vidya Mandal University)

MINI PROJECT REPORT

ON

STUDENT RECORD SEARCH SYSTEM

Submitted in partial fulfillment of the requirements for

Design and analysis of Algorithms

Submitted By

Student Name: Patel Dhairya R

Enrollment No.: 12402080601028

Class: IT-A [1B8]

Academic Year: 2026 – 2027

CERTIFICATE

This is to certify that the mini project report entitled “Student Record Search System” has been prepared and submitted by the student named below, in partial fulfillment of the academic requirements of [Course / Subject Name].

Student Name : Patel Dhairya

Enrollment No.: 12402080601028

The work presented in this report is carried out under the guidance of the undersigned. To the best of our knowledge, this report is suitable for submission.

Faculty Guide

Name: Minakshi patil

Signature: _____

Date: _____

Head of Department

Signature: _____

Date : _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Faculty Guide for valuable guidance, encouragement, and support throughout the development of this mini project. I am thankful to the Head of Department and the college for providing the necessary facilities and learning environment.

I also thank my teachers, classmates, and everyone who helped me directly or indirectly in completing this project successfully.

ABSTRACT

The Student Record Search System is a simple Python-based application designed to store student details and search records efficiently. The system maintains information such as student ID, name, course, and marks. It provides two searching techniques: Linear Search and Binary Search. Linear Search checks records one by one and is suitable for unsorted data, while Binary Search works on sorted data and reduces the search space by half at each step. This project demonstrates the use of Python lists, dictionaries, functions, sorting, and basic algorithm analysis. The system is intended for learning and small-scale record management.

TABLE OF CONTENTS

1. Introduction and Problem Statement
2. Objectives and Scope
3. Software and Hardware Requirements
4. System Design and Workflow
5. Python Data Structures and Functions
6. Linear Search and Binary Search Algorithms
7. Sample Input and Output
8. Testing and Results
9. Limitations and Future Enhancements
10. Conclusion
11. References

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

Student information is commonly maintained by educational institutions. Finding a particular student's details manually from a large register can take time and may lead to errors. A computerized search system can make this task faster and more organized.

This mini project uses Python to store student records and search them using Linear Search and Binary Search. It is a basic console-based application suitable for demonstrating searching algorithms.

1.2 Problem Statement

To develop a simple program that stores student records and allows a user to find a student by ID. The program should support searching in unsorted records using Linear Search and in sorted records using Binary Search, and display the matching student details or a message when the record is not found.

2. OBJECTIVES AND SCOPE

2.1 Objectives

- To create and manage a small collection of student records.
- To search a student record using the student ID.
- To implement Linear Search and Binary Search in Python.
- To compare the basic time complexity of both algorithms.
- To display clear results for found and not-found records.

2.2 Scope

The project is intended for a small educational demonstration. It supports record entry, display, and ID-based searching. It does not include a database, login system, or multi-user access. Records are kept in program memory and are lost when the program closes unless file storage is added.

3. SOFTWARE AND HARDWARE REQUIREMENTS

3.1 Software Requirements

- Operating System: Windows, Linux, or macOS
- Programming Language: Python 3.x
- Editor / IDE: IDLE, VS Code, PyCharm, or any text editor
- Libraries: Python standard library only

3.2 Hardware Requirements

- Processor: Any modern basic processor
- RAM: 2 GB or more (recommended)
- Storage: A few MB of free space
- Input / Output: Keyboard and monitor

4. SYSTEM DESIGN AND WORKFLOW

4.1 System Overview

The system accepts student details, stores them in a list, and lets the user choose a search method. Linear Search examines each record sequentially. Binary Search first sorts records by student ID and then repeatedly compares the target ID with the middle record.

4.2 Workflow

- Start the program.
- Create or load the student records.
- Display the menu: add/display/search/exit.
- For search, enter the student ID.
- Choose Linear Search or Binary Search.
- Run the selected algorithm.
- Display the student details if found; otherwise display “Record not found.”
- Return to the menu or exit.

4.3 Textual Flow Diagram

START → ENTER / LOAD RECORDS → SELECT SEARCH METHOD

→ ENTER STUDENT ID → SEARCH → FOUND? → DISPLAY RECORD

↘ NOT FOUND → DISPLAY MESSAGE → END / MENU

5. PYTHON DATA STRUCTURES AND FUNCTIONS

5.1 Data Structures

A list stores all student records. Each student record is represented by a dictionary containing ID, name, course, and marks. Example:

```
students = [
  {'id': 101, 'name': 'Asha', 'course': 'BCA', 'marks': 82},
  {'id': 102, 'name': 'Ravi', 'course': 'BCA', 'marks': 76}
]
```

5.2 Main Functions

- add_student(): accepts a student's details and adds a dictionary to the list.
- display_students(): prints all stored records.
- linear_search(): checks each record for the requested ID.
- binary_search(): searches the ID in a list sorted by ID.
- main(): displays the menu and calls the required function.

6. LINEAR SEARCH AND BINARY SEARCH ALGORITHMS

6.1 Linear Search

Linear Search starts from the first record and compares each student ID with the target ID. If a match is found, its position is returned. If the list ends without a match, the result is -1.

Algorithm:
1. Start from index 0.
2. Compare each record ID with the target ID.
3. If equal, return the record.
4. If no match exists, return None.

Time Complexity: Best case O(1); average and worst case O(n). Space Complexity: O(1) extra space.

6.2 Binary Search

Binary Search requires the records to be sorted by student ID. It compares the target with the middle record. If the target is smaller, it searches the left half; otherwise, it searches the right half.

Algorithm:
1. Sort records by student ID.
2. Set low = 0 and high = n − 1.
3. Find mid = (low + high) // 2.
4. If middle ID equals target, return the record.
5. If target is smaller, set high = mid − 1; otherwise set low = mid + 1.
6. Repeat while low <= high. If not found, return None.
Time Complexity: Best case O(1); average and worst case O(log n). Sorting before search may take O(n log n). Space Complexity: O(1) extra space for iterative search.

6.3 Python Implementation

```
students = [
   {"id": 101, "name": "Asha", "course": "BCA", "marks": 82},
   {"id": 104, "name": "Karan", "course": "BCA", "marks": 75},
   {"id": 102, "name": "Ravi", "course": "BCA", "marks": 76},
   {"id": 103, "name": "Neha", "course": "BCA", "marks": 91}
]

def linear_search(records, target):
   for student in records:
      if student["id"] == target:
         return student
   return None

def binary_search(records, target):
   records = sorted(records, key=lambda student: student["id"])
   low = 0
   high = len(records) - 1

   while low <= high:
      mid = (low + high) // 2
      if records[mid]["id"] == target:
         return records[mid]
      elif target < records[mid]["id"]:
         high = mid - 1
      else:
         low = mid + 1
   return None

def display_result(student):
   if student:
      print("Record found:", student)
   else:
      print("Record not found.")

student_id = int(input("Enter student ID to search: "))
choice = input("Choose search (L for Linear, B for Binary): ").upper()
```

```
if choice == "L":
    result = linear_search(students, student_id)
elif choice == "B":
    result = binary_search(students, student_id)
else:
    result = None
    print("Invalid search choice.")

if choice in ("L", "B"):
    display_result(result)
```

7. SAMPLE INPUT AND OUTPUT

7.1 Sample Input

Student records: 101 – Asha – BCA – 82; 104 – Karan – BCA – 75; 102 – Ravi – BCA – 76; 103 – Neha – BCA – 91.

7.2 Output: Record Found

```
Enter student ID to search: 103
Choose search (L for Linear, B for Binary): B
Record found: {'id': 103, 'name': 'Neha', 'course': 'BCA', 'marks': 91}
```

7.3 Output: Record Not Found

```
Enter student ID to search: 999
Choose search (L for Linear, B for Binary): L
Record not found.
```

8. TESTING AND RESULTS

The program is tested with different student IDs and search choices to verify correctness.

Test Case	Input	Expected Result	Status
1	ID 103, Linear Search	Neha's record displayed	Pass
2	ID 103, Binary Search	Neha's record displayed	Pass
3	ID 999, Linear Search	Record not found	Pass
4	ID 101, Binary Search	Asha's record displayed	Pass
5	Invalid choice X	Invalid search choice message	Pass

Result: The implementation correctly returns matching records for valid IDs and reports when an ID is absent. Binary Search operates on sorted records and is more efficient for large sorted datasets.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Limitations

- Data is stored only in memory and is not permanent.
- The program is suitable for small datasets and a single user.
- Binary Search requires sorted records.
- The current version searches only by student ID.
- Input validation and security features are basic.

9.2 Future Enhancements

- Add file or database storage such as CSV, SQLite, or MySQL.
- Create a graphical user interface using Tkinter.
- Add update and delete operations.
- Search by student name, course, or enrollment number.
- Add login, role-based access, and input validation.

10. CONCLUSION

The Student Record Search System demonstrates how Python can be used to organize student details and search records. Linear Search is simple and works with unsorted data, whereas Binary Search is faster for sorted data. The project provides practical understanding of lists, dictionaries, functions, sorting, searching algorithms, and time complexity. It can be extended into a more complete student management application by adding persistent storage and a user interface.

11. REFERENCES

- Python Software Foundation, Python 3 Documentation: https://docs.python.org/3/
- Cormen, Leiserson, Rivest, and Stein, Introduction to Algorithms, MIT Press.
- Course notes and classroom material on searching algorithms and algorithm analysis.